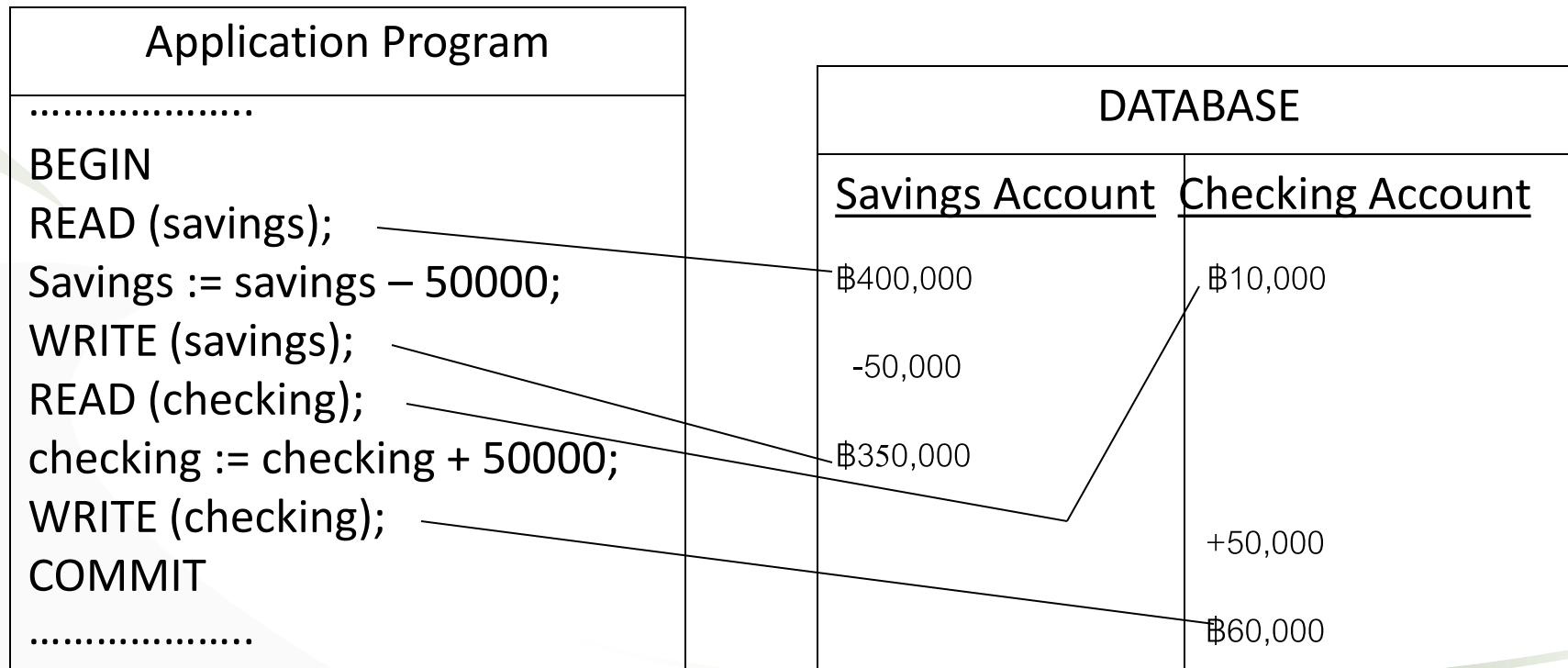


Transaction, Recovery and Concurrency Control

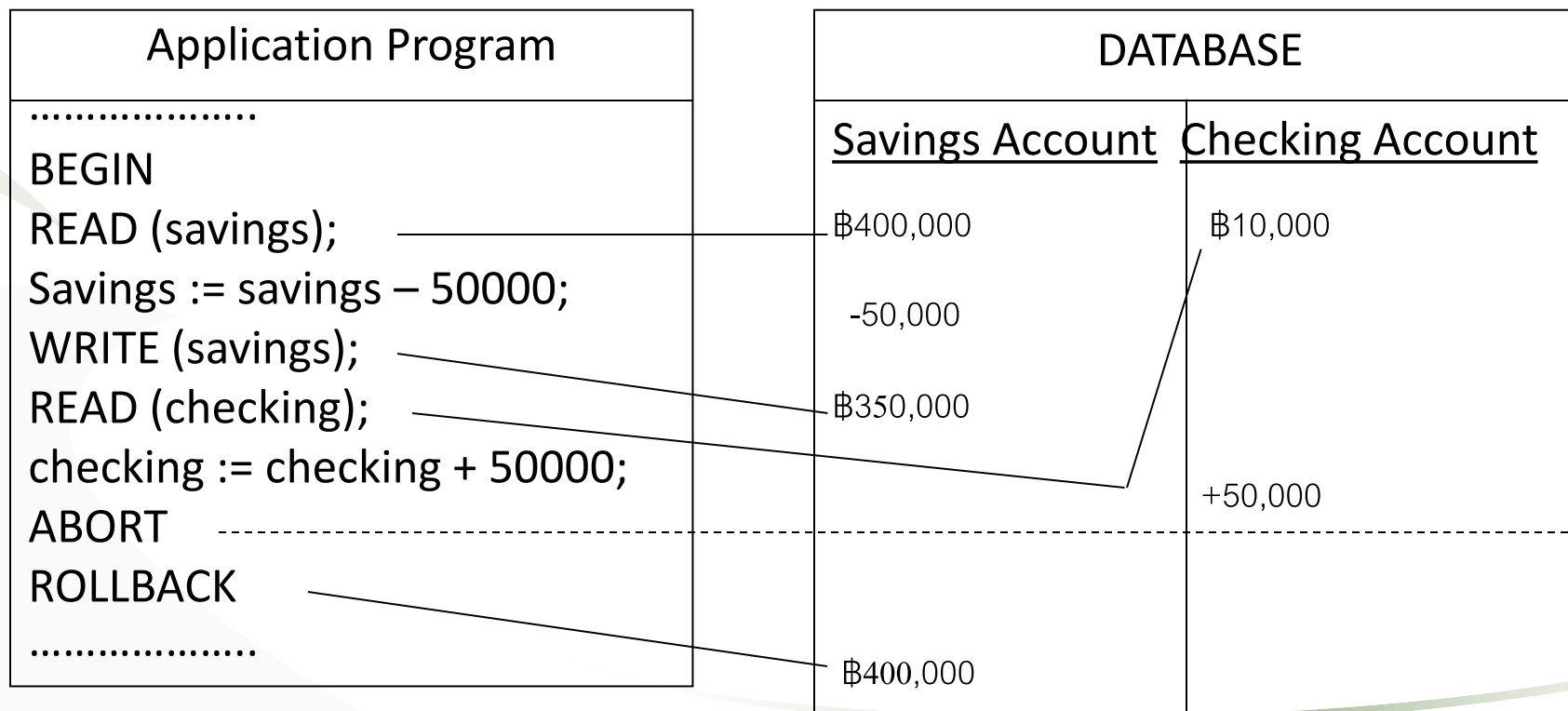
Transaction Concept

- A transaction is a unit of program execution that access and possibly updates various data items
- Example of a successful transaction



Transaction Concept (cont.)

- Example of an aborted transaction



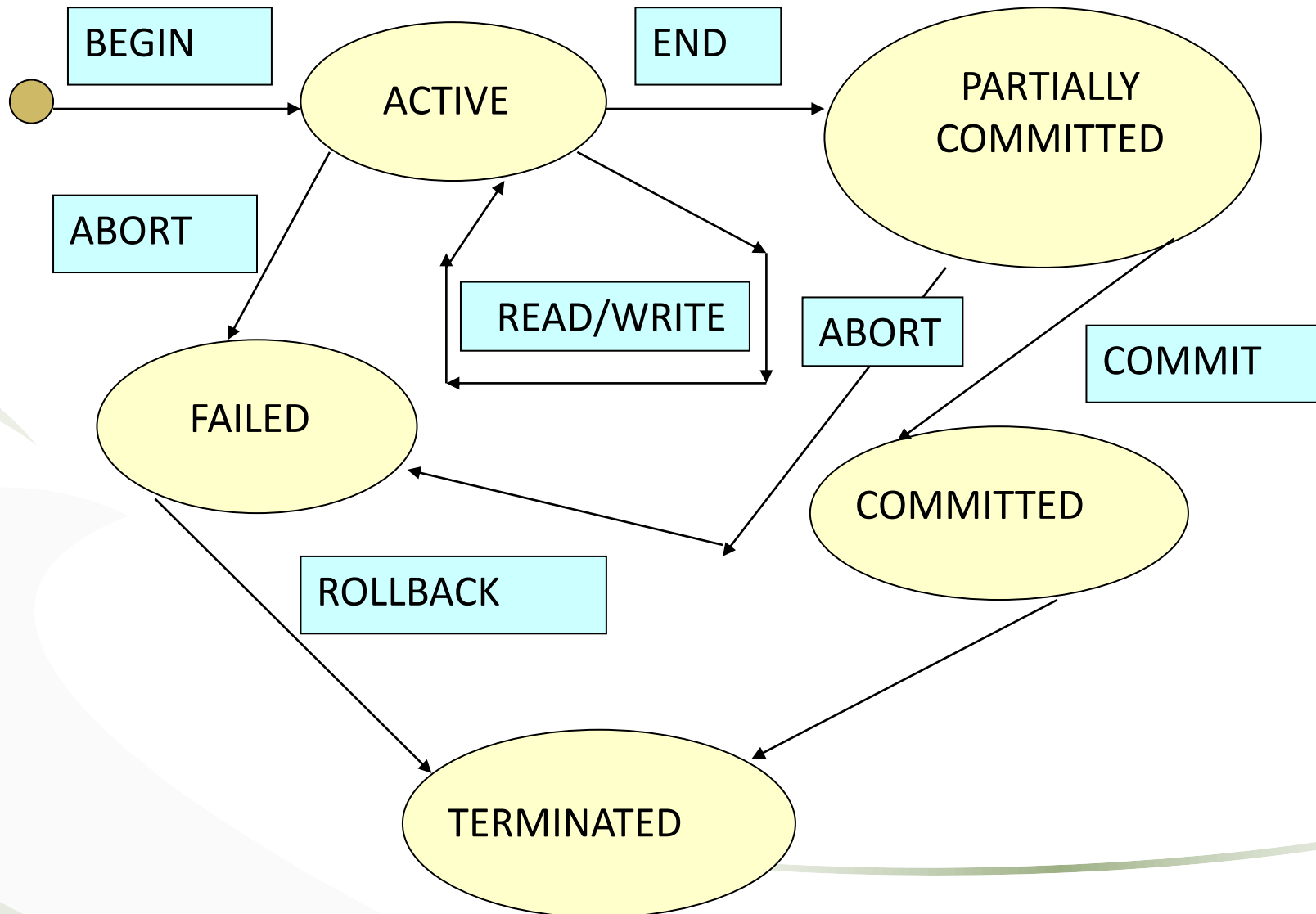
Transaction Properties

- Properties of transactions : **ACID** properties
- **Atomicity** : if it execute all of its database updates or none at all (all or nothing)
- **Consistency** : preserve database consistency
- **Isolation** : multiple transaction execute concurrency, each transaction is unaware of other transaction
- **Durability** : database update by successfully completed transactions must be permanent in the database

Transaction States

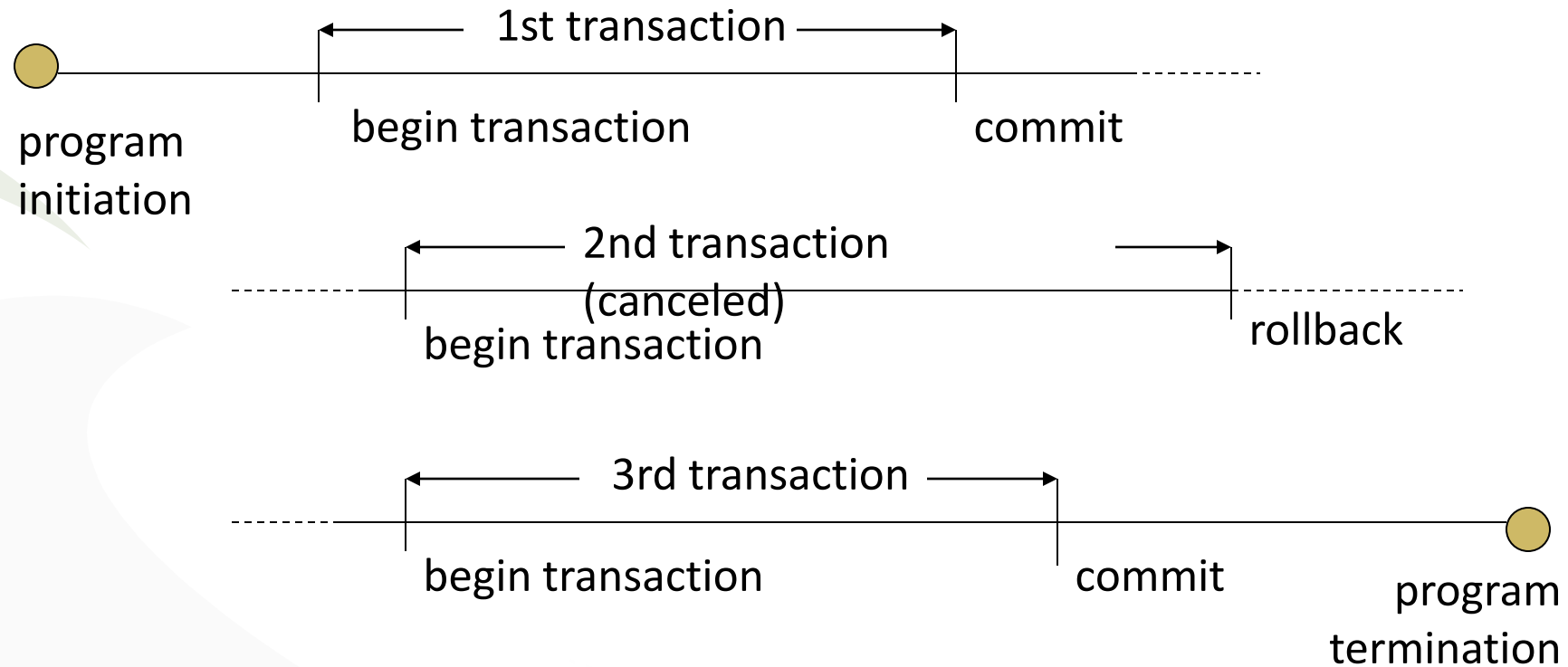
- BEGIN : transaction enters **active state**
- READ : transaction remains in **active state**
- WRITE : transaction remains in **active state**
- END : transaction proceeds to partially **committed state**
- COMMIT : transaction moves to **committed state** and finally reaches the **terminated state**
- ABORT : transaction proceeds to failed state from either **active state** or **partially committed state**
- ROLLBACK : transaction reaches the **terminated state**

Transaction States (cont.)



Recovery

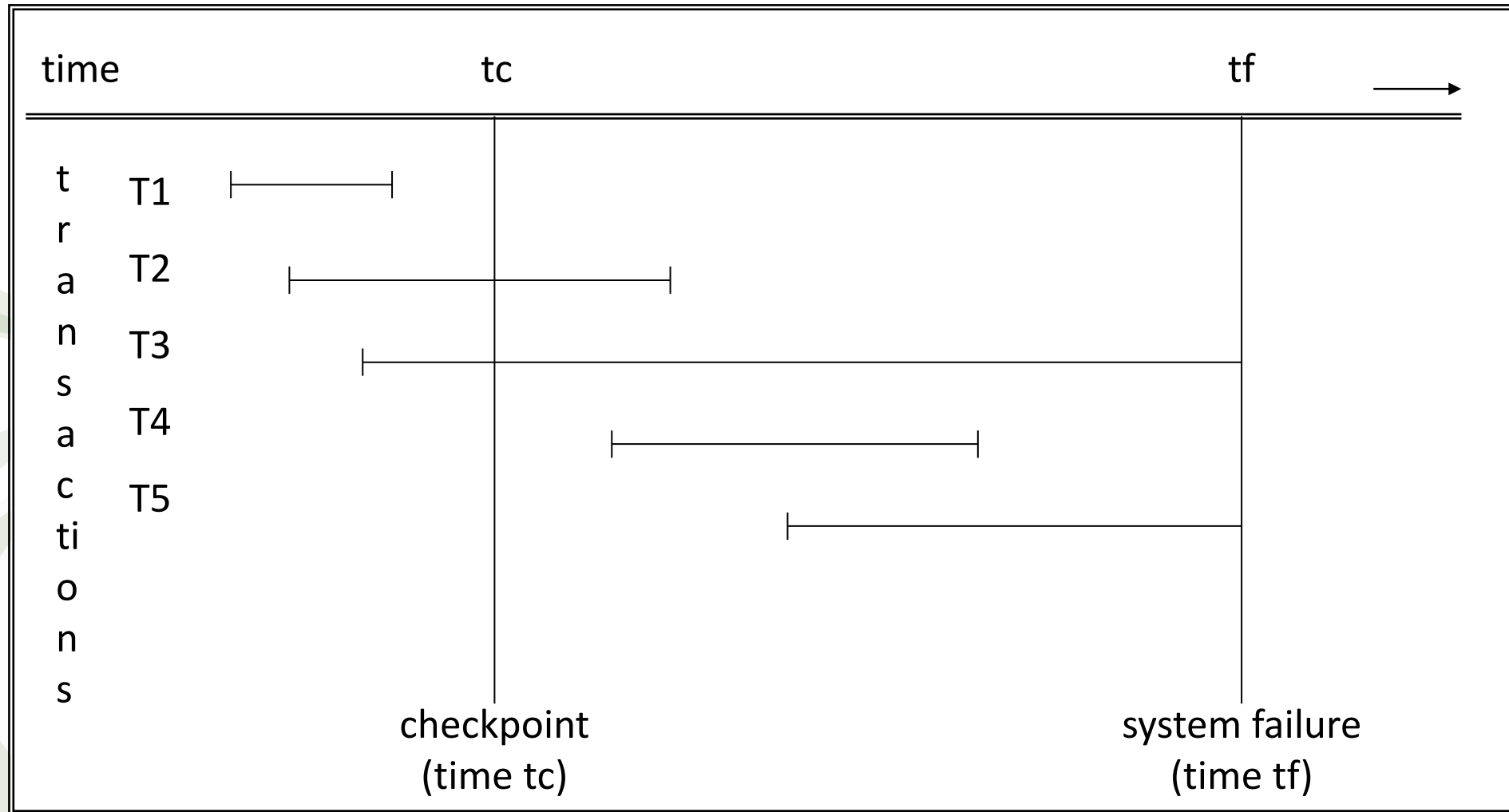
- Program execution is a sequence of transactions



Recovery (cont.)

- Failures :
 - system failures e.g. power outage (soft crash) → system recovery (log-based recovery)
 - media failures e.g. head cash on the disk (hard crash) → media recovery

System Recovery



System Recovery (cont.)

- At restart time create UNDO/REDO list
 1. Set the UNDO list equal to the list of all transactions given in the most recent checkpoint record; set the REDO list to empty.
 2. Search forward through the log, start from the checkpoint record.
 3. If a BEGIN TRANSACTION log entry is found for transaction T, add T to the UNDO list.
 4. If a COMMIT log entry is found for transaction T, move T from the UNDO list to the REDO list.
 5. When the end of the log is reached, the UNDO and REDO lists identify.

Two-phase commit

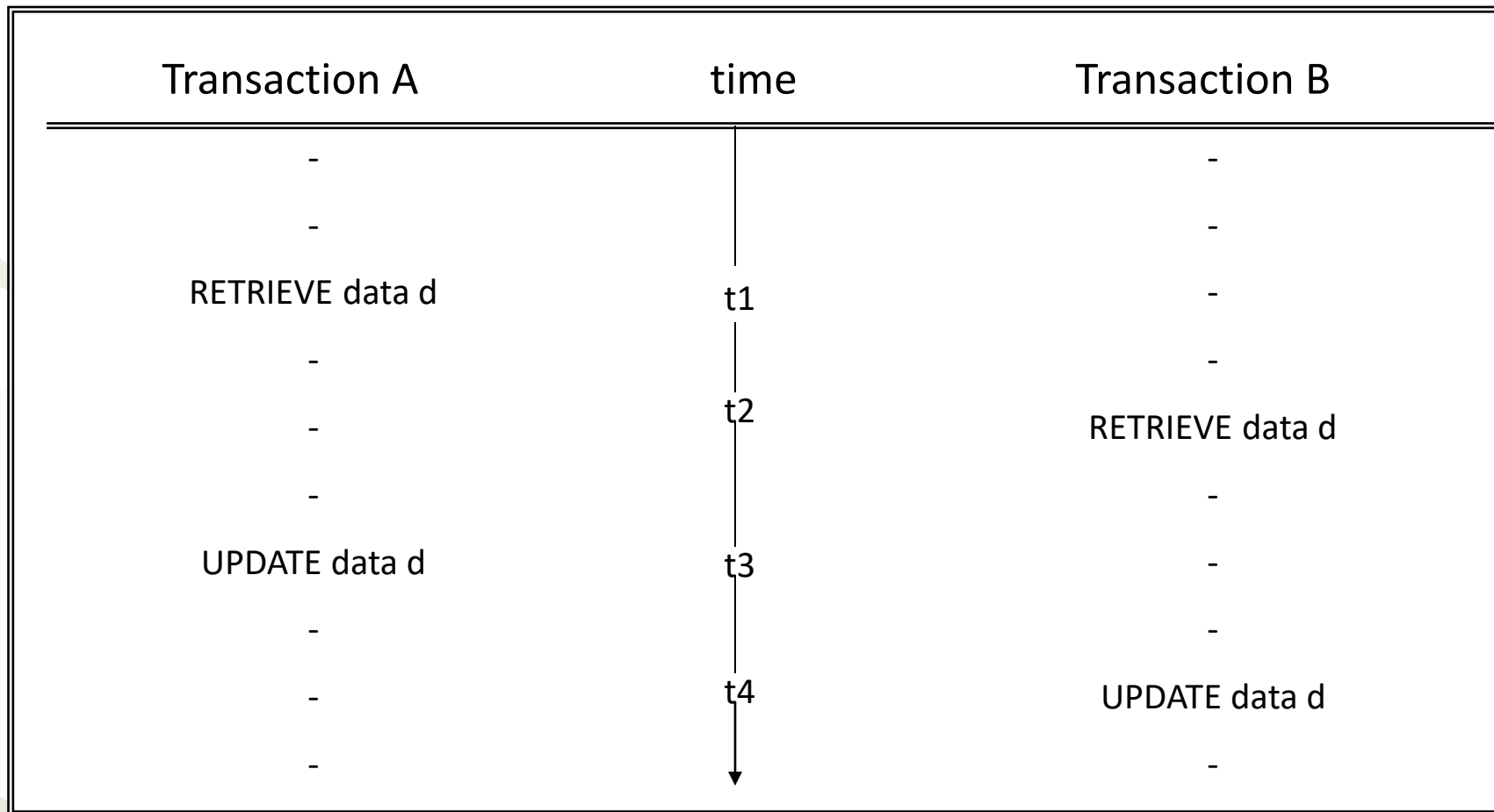
- Two-phase commit is a protocol use in the system that permit transactions to interact with two or more distinct resource manager e.g. two different DBMSs
- To maintain the transaction atomicity property

Two-phase Commit (cont.)

- The two phases are :-
 1. the prepare phase : the coordinator instructs all participants to “get ready to go either way”
 2. the commit phase : in which assuming all participants responded satisfactorily during the prepare phase, the coordinator the instruct all participants to perform the actual commit (or the actual rollback otherwise)

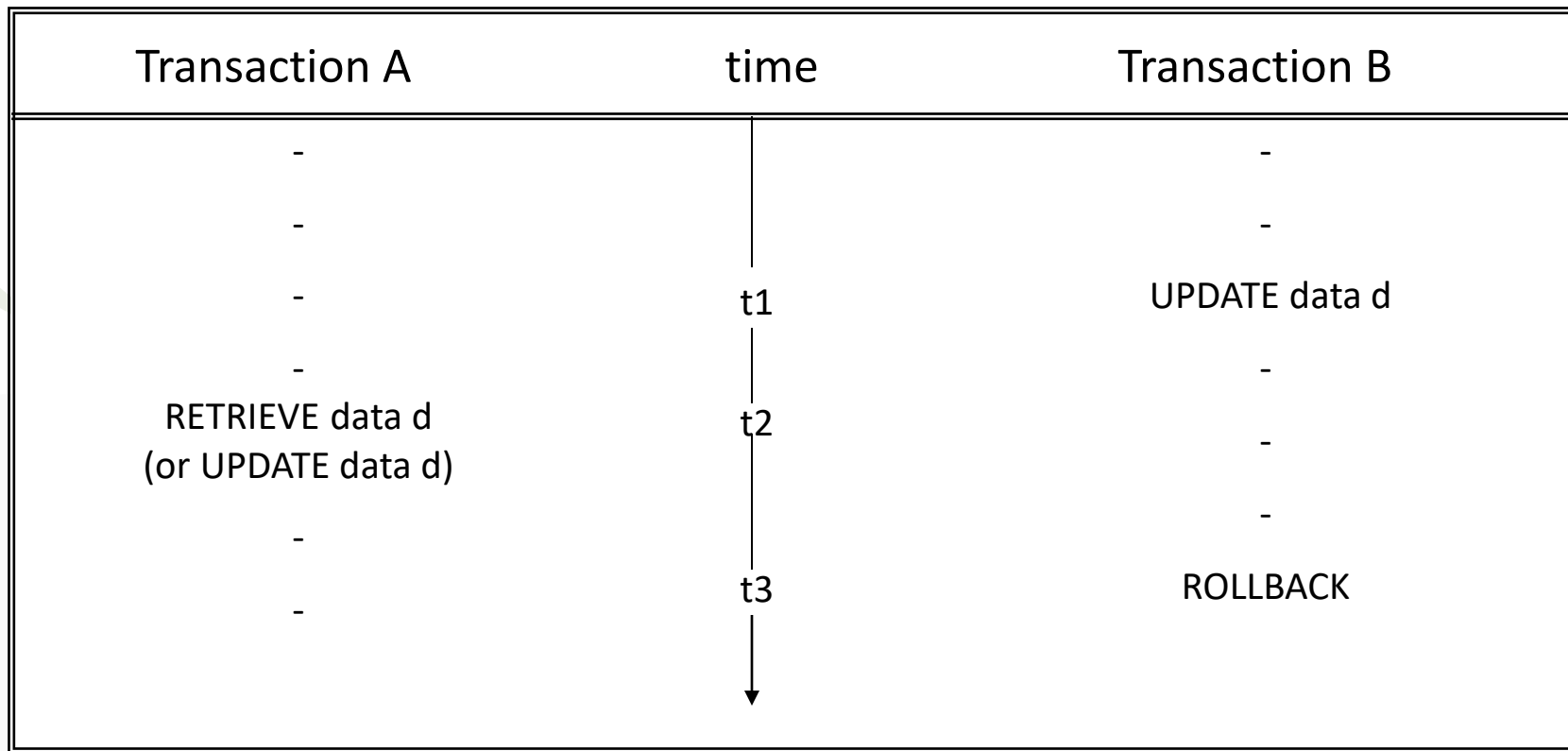
Concurrency Control

- Three concurrency problems
- The lost update problem



Concurrency Control (cont.)

- The uncommitted dependency problem



Concurrency Control (cont.)

- The inconsistent analysis problem

ACC 1	40	ACC 2	50	ACC 3	30
Transaction A		time	Transaction B		
RETRIEVE ACC 1 sum = sum + ACC1		t1	-		
			-		
RETRIEVE ACC 2 sum = sum + ACC 2		t2	-		
			-		
-	-		-		
-	-	t3	RETRIEVE ACC 3		
-	-	t4	UPDATE ACC 3 30 → 20		
-	-	t5	RETRIEVE ACC 1		
-	-	t6	UPDATE ACC 1 40 → 50		
-	-	t7	COMMIT		
-	-				
RETRIEVE ACC 3 (sum = sum + ACC3 =?)		t8			
		↓			

Lock-based Resolution

- Two basic types of locks :
 1. Exclusive lock or write lock : indicated as x-lock
 2. Shared lock or read lock : indicated as s-lock
- Matrix for locking

	X	S	-
X	N	N	Y
S	N	Y	Y
-	Y	Y	Y

Simple Locking

INVENTORY RECORDS IN DATATBASE

Order Entry in Chicago

(order database textbook 70 copies)

1. REQUEST LOCK (DB)
2. LOCK (DB)
3. READ (DB)

5. (DB) := (DB)-70
6. WRITE (DB)
7. UNLOCK (DB)

DB

200

130

70

Order Entry in Boston

(order database textbook 60 copies)

4. REQUEST LOCK (DB)

8. LOCK (DB)
9. READ (DB)
10. (DB) := (DB) -60
11. WRITE (DB)
12. UNLOCK (DB)

Simple Locking (cont.)

- Initial value (D1) 100, (D2) 150

Operation in T1 and T2

T1	T2
LOCK-S (D2)	LOCK-S (D1)
READ (D2)	READ (D1)
UNLOCK (D2)	UNLOCK (D1)
LOCK-X (D1)	LOCK-X (D2)
READ (D1)	READ (D2)
$(D1) := (D1) + (D2)$	$(D2) := (D2) + (D1)$
WRITE (D1)	WRITE (D2)
UNLOCK (D1)	UNLOCK (D2)
...	...

Simple Locking (cont.)

TIME	T1 Chicago order	T2 Boston order	DB
	DB 70 Copies	DB 60 Copies	
1	BEGIN T1		200
2	LOCK-X (DB)	BEGIN T2	200
3	READ (DB)	LOCK-X (DB)	200
4	(DB) := (DB) - 70	 wait	200
5	WRITE (DB)	 wait	130
6	UNLOCK (DB)	 wait	130
7	COMMIT	READ (DB)	130
8	END	(DB) := (DB) - 60	130
9		WRITE (DB)	70
10		UNLOCK (DB)	70
11		COMMIT	70
12		END	70